Partial derivatives and the gradient: which way is uphill when a model has many numbers to tune

Many models have billions of adjustable numbers. Which way should each one move to lower the loss?

Mathematics for AI and Machine Learning | Session 8 of 18 | Calculus and optimisation
KLAi Professional Learning | Jotham Martin Wambi

Today in one view: what you will leave with

By the end of today you can



Compute a partial derivative of a function of two inputs by changing one input and holding the other fixed



Collect the partial derivatives into the gradient vector and say which way is steepest uphill on a contour plot



Compute the gradient of the loss of a line fit (intercept and slope) by hand and check it with numbers

Your two hours: many inputs, the gradient, then the lab

0:00	10	Recall, and a real question from AI that today answers
0:10	35	Concept: a picture, then a hand example
0:45	10	Break
0:55	45	Lab in Python or a spreadsheet: predict, run, change one thing
1:40	15	Where it appears in ChatGPT, Gemini, Claude; exit check
1:55	5	Save the work

Pictures and hand examples first, then the lab on the same numbers in Python or a spreadsheet.

Three questions from last time

1. What does the derivative of a function tell you at one point?

2. The slope of the loss at w = 1 is -18.5. Which way do you move w, and why?

3. How do you check a derivative worked out by hand?

Answer aloud first. Then we check together.

Answers to the three questions from Session 7

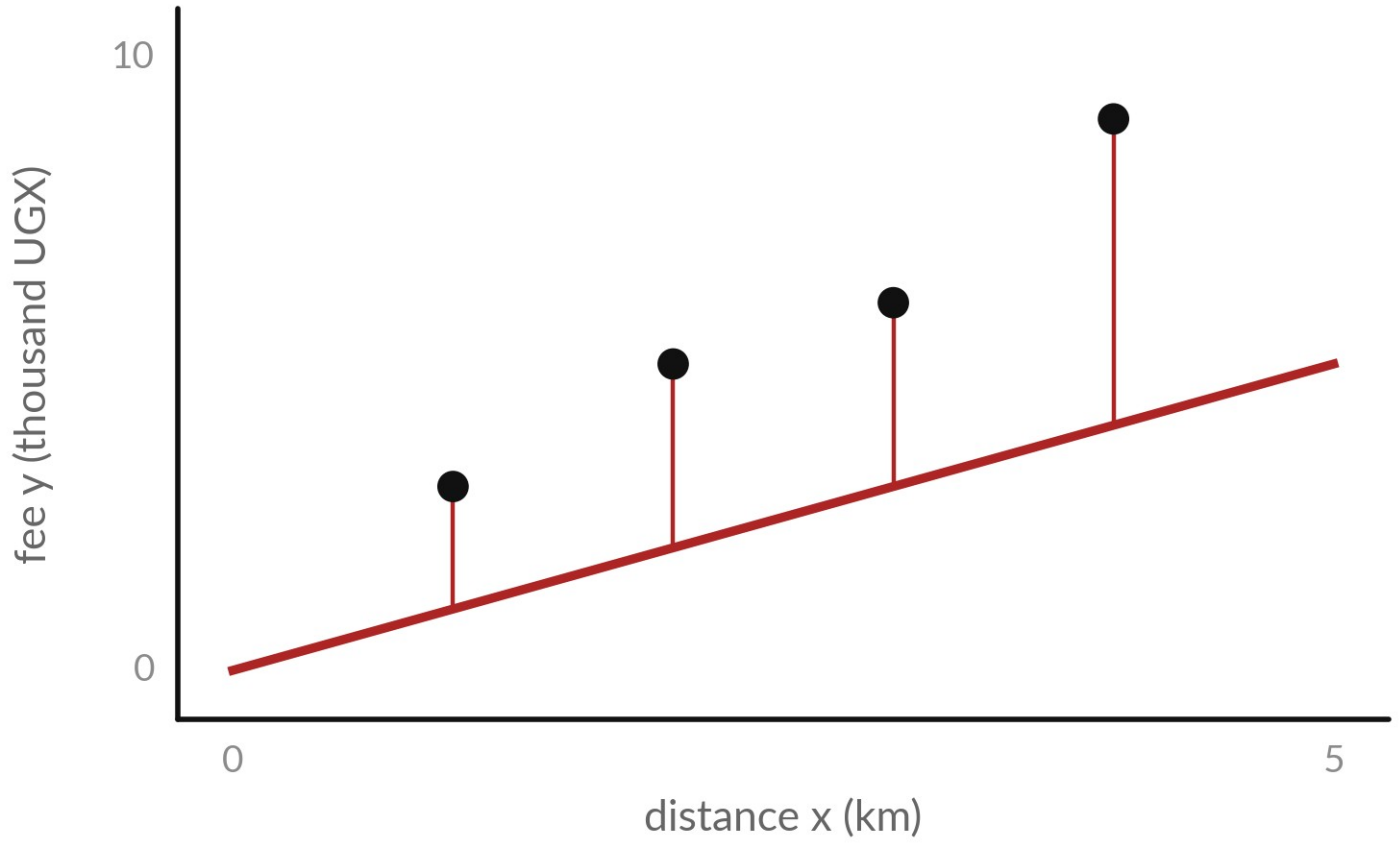
Question	Short answer
What does the derivative of a function tell you at one point?	The slope of the tangent line there: how fast the output changes for a small change in the input.
The slope of the loss at w = 1 is -18.5. Which way do you move w, and why?	Up. A negative slope means the loss falls as w grows, so you raise w.
How do you check a derivative worked out by hand?	Nudge the input by a tiny h and compute (f(x + h) minus f(x - h)) divided by 2h. It should match the formula to a few decimals.

If one answer surprised you, look back at Session 7 before the lab.

Words we use today

	Word	What it means
	**Parameter**	A number inside a model that training can change. Our line y-hat = b + w x has two: the intercept b and the slope w.
	**Partial derivative**	The slope of a function when you change one input and hold the others fixed. Written ∂L/∂w and read "partial L, partial w".
	**Gradient**	The vector of all the partial derivatives, written as a column. It points where the function increases fastest. Symbol: nabla.
	**Contour plot**	A map of a function of two inputs. Each curve joins the points where the function has the same value.
	**Steepest increase**	The direction in which a function rises fastest. Its opposite, minus the gradient, is the direction of steepest decrease.

A line fit has two parameters, so its loss depends on two numbers



Last session the model was y-hat = w x. Now the line can also shift up or down: y-hat = b + w x.

Two parameters: the intercept b and the slope w. The red segments are the errors at w = 1 and b = 0.

The loss L(w, b) depends on both. L(1, 0) = 11.75. For w = 1.9 and b = 1 it is 0.175.

The loss of a line fit is a function of two inputs, w and b

Loss (mean squared error)

$$L(w, b) = \frac{1}{n} \sum_{i=1}^{n} (b + w\,x_i - y_i)^2$$

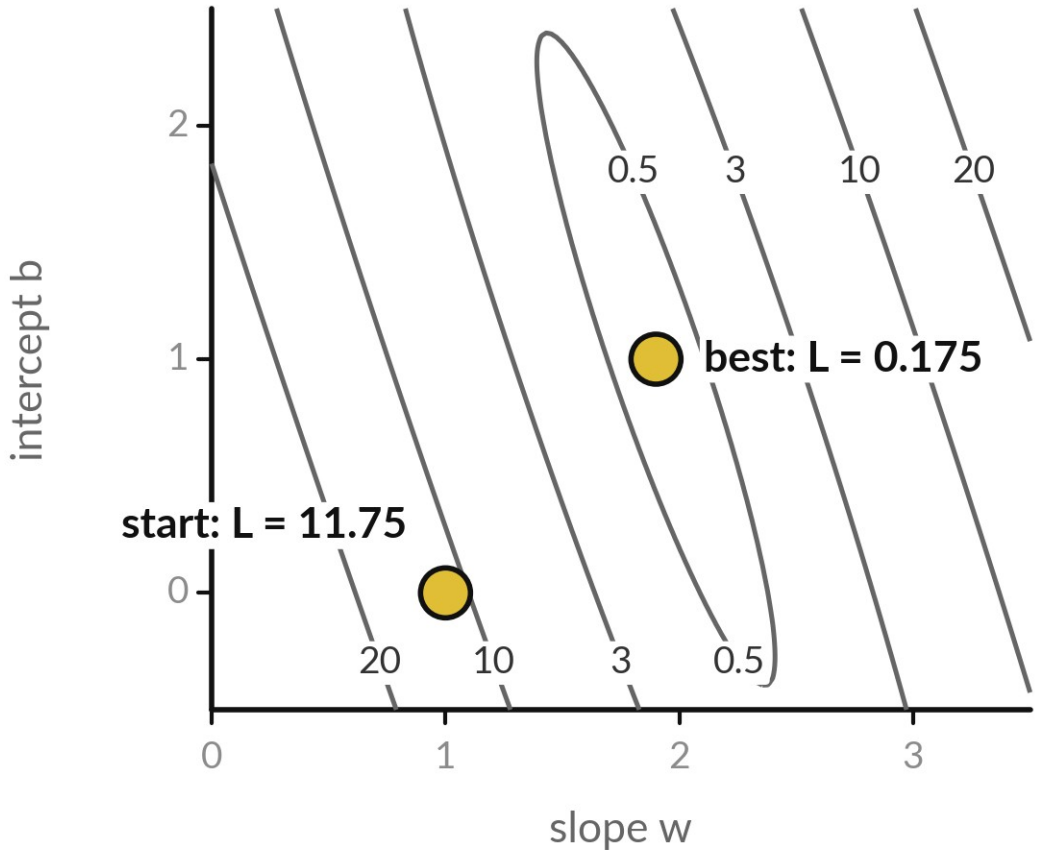
Our data at (1, 0)

$$L(1, 0) = (4 + 9 + 9 + 25)/4 = 11.75$$

Symbol	What it means	In our data
L(w, b)	The loss: the mean of the squared errors for one pair (w, b).	L(1, 0) = 11.75; L(1.9, 1) = 0.175
w, b	The two parameters: slope (fee per km) and intercept (fee at 0 km).	start: w = 1, b = 0
x_i, y_i, n	Distance and fee of delivery i, and the number of rows.	x = 1, 2, 3, 4; y = 3, 5, 6, 9; n = 4

The error of row i is e_i = b + w x_i - y_i. At (1, 0) the four errors are -2, -3, -3 and -5.

A contour plot is a map of the loss: each curve joins the pairs (w, b) with the same loss

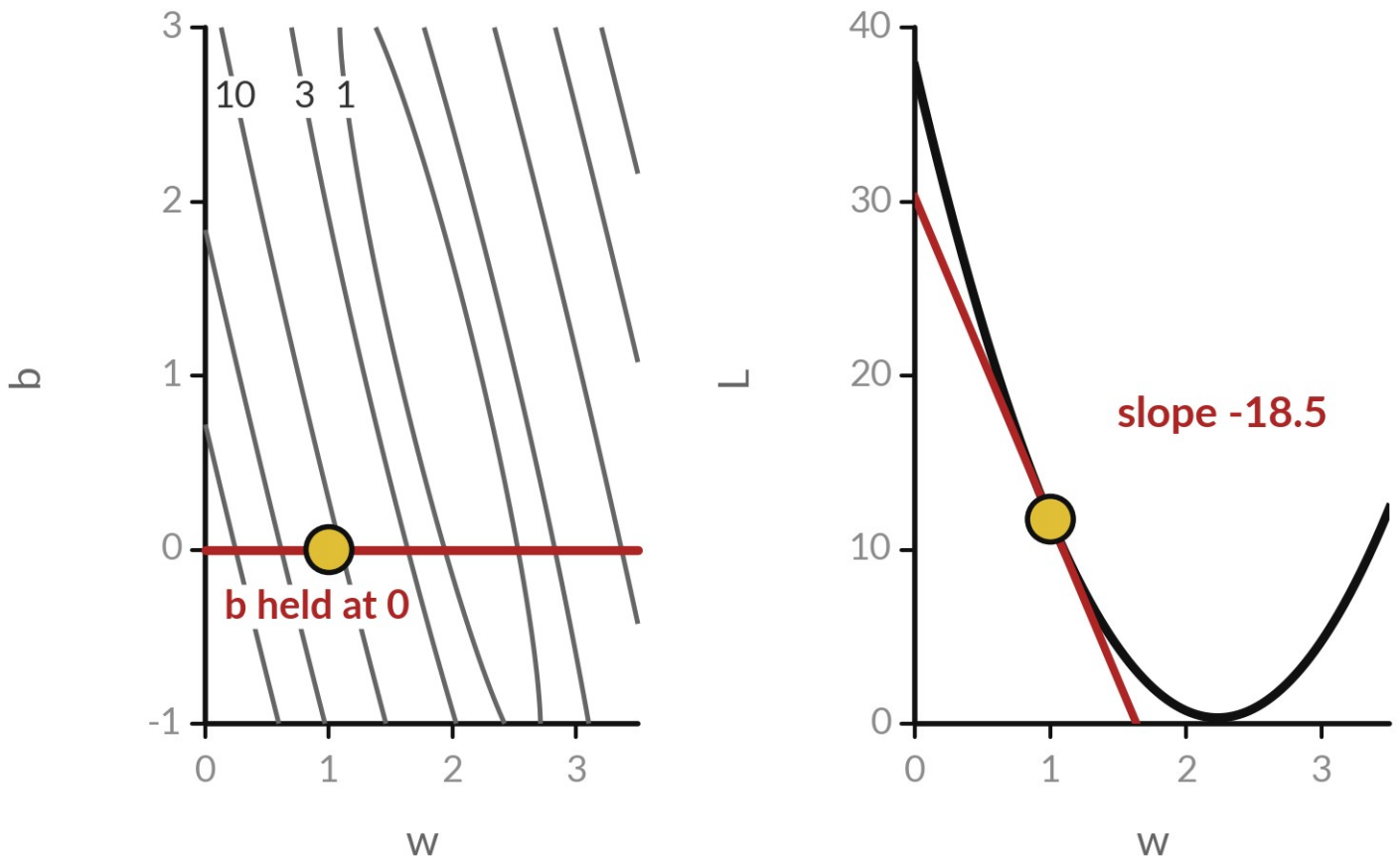


Each grey curve is a contour line, like a height line on a hiking map. The small numbers give the loss on that curve.

The start (1, 0) lies between the curves 20 and 10.

The best line, (1.9, 1), sits inside the small closed curve 0.5, where the loss is smallest.

A partial derivative is the slope of one slice: change one input and hold the other fixed



Hold b at 0 and change only w. The loss along the red horizontal line (left) is last session's curve (right).

Its slope at w = 1 is -18.5. That is the partial derivative of L with respect to w, written ∂L/∂w.

Hold w fixed and change only b to get the other partial derivative, ∂L/∂b.

Partial derivatives: differentiate with respect to one input and treat the others as constants

Function

$$f(x, y) = x^2 + 3y^2$$

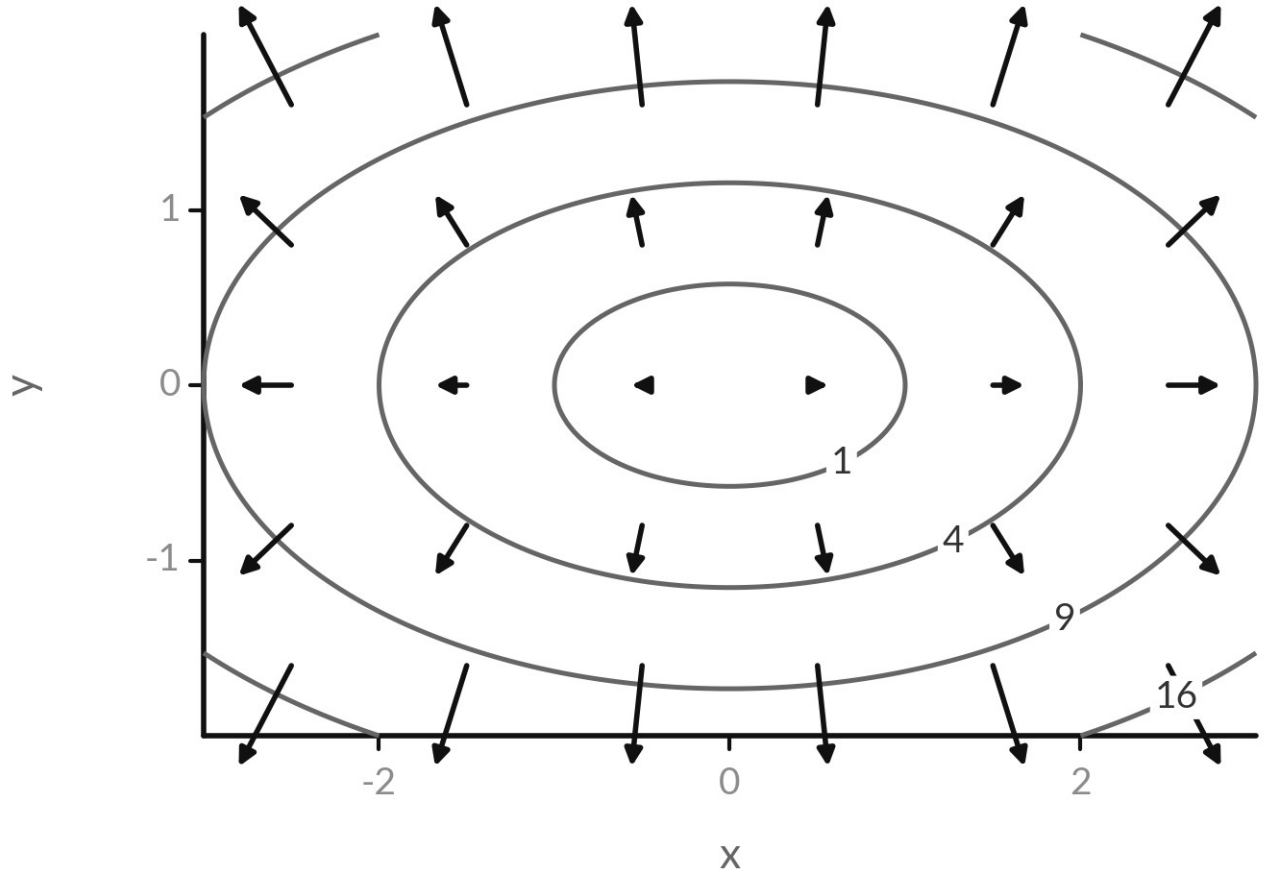
Partial derivatives

$$\frac{\partial f}{\partial x} = 2x \qquad \frac{\partial f}{\partial y} = 6y$$

Symbol	What it means	In our data
∂ (partial)	The partial derivative symbol, read "partial". All other inputs are held fixed.	∂f/∂x with y held fixed
∂f/∂x = 2x	Treat 3 y squared as a constant. Its slope in x is 0. The power rule on x squared gives 2x.	at (2, 1): 4
∂f/∂y = 6y	Treat x squared as a constant. The power rule on 3 y squared gives 6y.	at (2, 1): 6

Check at (2, 1): f(2.001, 1) minus f(1.999, 1), divided by 0.002, is 4.000. And in y it is 6.000.

The gradient points in the direction of steepest increase, at right angles to the contour lines



The function is f(x, y) = x squared + 3 y squared. Each black arrow is the gradient at its point.

Every arrow points uphill and crosses the contour lines at a right angle.

Long arrows mean a steep slope. The arrows shrink towards the centre, where the bowl is flat.

The gradient is the column of all the partial derivatives

Gradient (a column vector)

$$\nabla f = \begin{pmatrix} \partial f/\partial x \\ \partial f/\partial y \end{pmatrix}$$

At (2, 1)

$$\nabla f(2, 1) = \begin{pmatrix} 4 \\ 6 \end{pmatrix}$$

Symbol	What it means	In our data
nabla f	Read "nabla f" or "grad f". One entry for each input, stood up as a column (Session 3). In text we write (4, 6).	(4, 6) at (2, 1)
length of nabla f	The steepness: the slope in the steepest direction. It is the L2 norm from Session 2.	square root of 16 + 36 = 7.21
minus nabla f	The direction of steepest decrease. Machine learning walks this way.	(-4, -6)

Uphill direction as a unit vector (length 1): (4, 6) divided by 7.21 is (0.55, 0.83). The slope in that direction is 7.21.

The gradient of the line-fit loss: one partial derivative for w and one for b

Slope of the loss in w

$$\frac{\partial L}{\partial w} = \frac{2}{n} \sum_{i=1}^{n} x_i \, e_i$$

Slope of the loss in b

$$\frac{\partial L}{\partial b} = \frac{2}{n} \sum_{i=1}^{n} e_i$$

Symbol	What it means	In our data
e_i	The error of row i: prediction minus actual, b + w x_i - y_i.	at (1, 0): -2, -3, -3, -5
x_i e_i	Each error times its input. It enters the w slope because w multiplies x_i.	-2, -6, -9, -20
sum e_i	In the b slope each error counts once, because b is added on its own.	-13

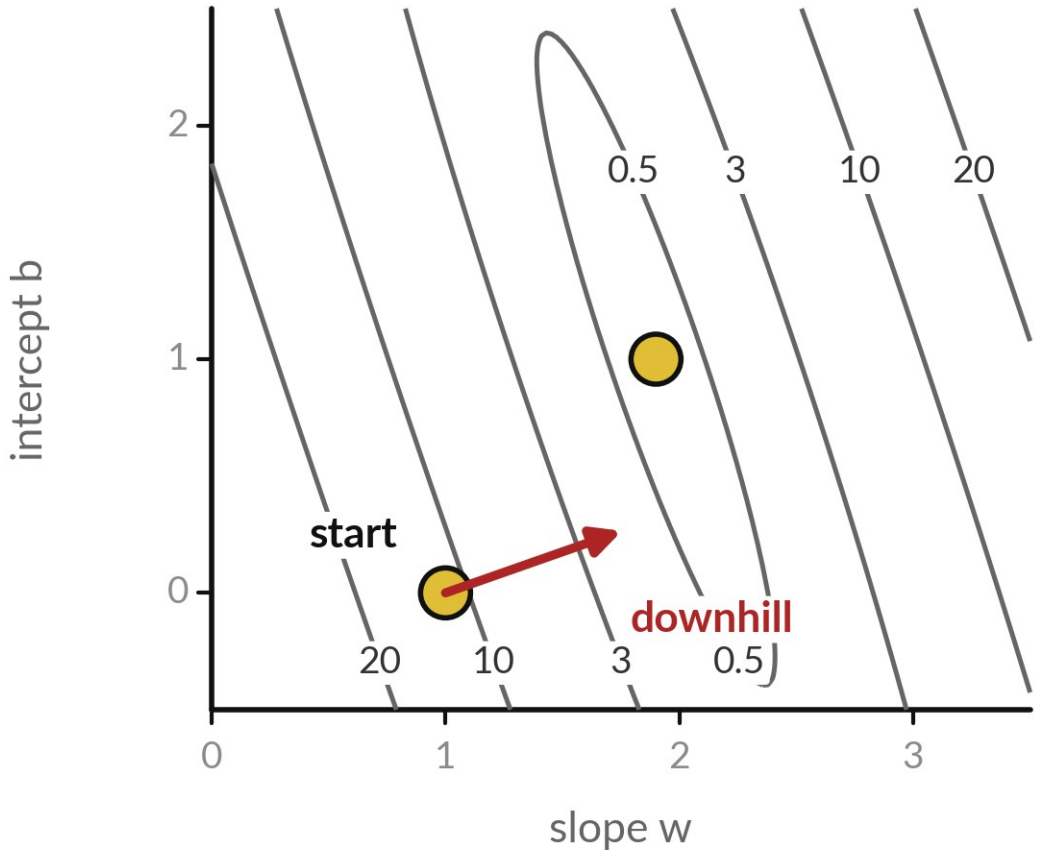
The 2 / n, the x_i and the 1 in front of e_i all come from the chain rule (Session 9).

Worked by hand: at w = 1 and b = 0 the gradient is (-18.5, -6.5)

Row i	Distance x	Fee y	Prediction b + w x	Error e	x times e
1	1	3	1	-2	-2
2	2	5	2	-3	-6
3	3	6	3	-3	-9
4	4	9	4	-5	-20
Sum				-13	-37

∂L/∂w = 2/4 times (-37) = -18.5. ∂L/∂b = 2/4 times (-13) = -6.5. Check: nudging w and b by 0.0001 each way gives -18.500 and -6.500.

Downhill is minus the gradient: at (1, 0) the loss falls fastest towards a larger w and a larger b

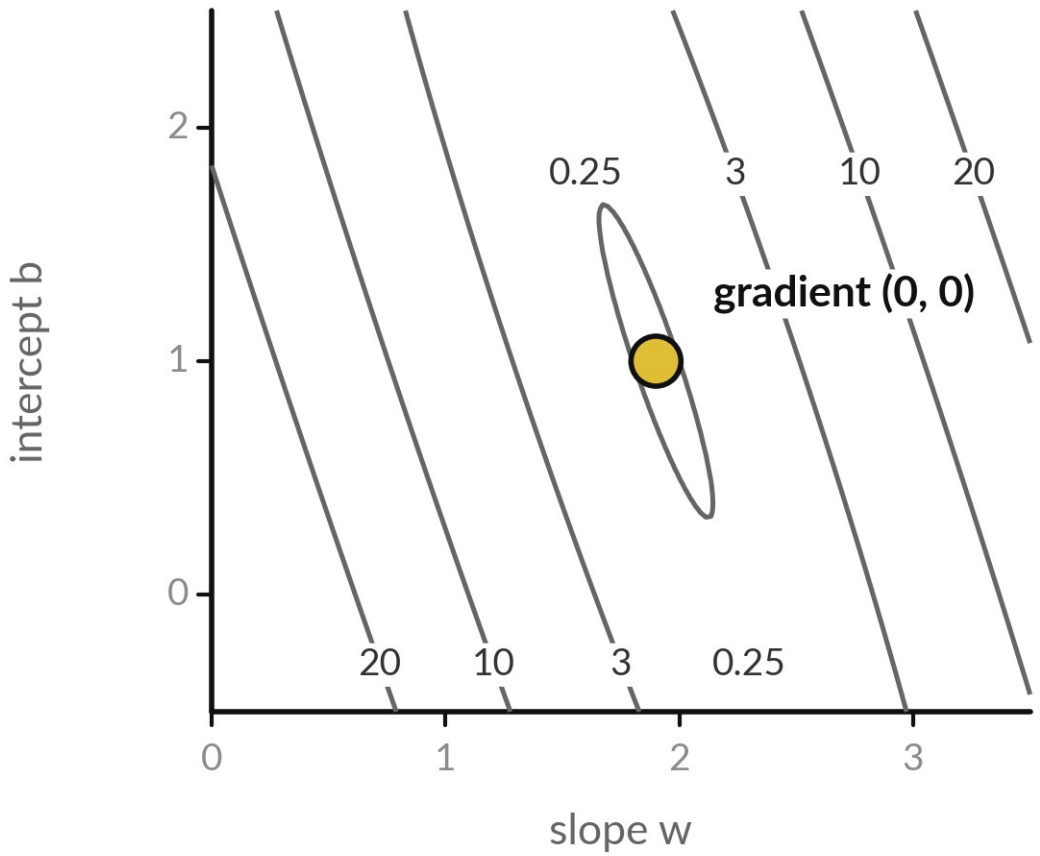


At the start the gradient is (-18.5, -6.5). The red arrow is minus the gradient, shortened to length 0.8.

Both entries are negative, so both w and b should go up. As a unit vector the direction is (0.943, 0.331).

The steepest slope there is the length of the gradient: 19.6 loss units per unit of step.

At the best fit the gradient is zero: the loss is flat in every direction



At w = 1.9 and b = 1 the four errors are -0.1, -0.2, 0.7 and -0.4.

Errors sum to 0, so $\partial L/\partial b = 0$. x times error sums to -0.1 - 0.4 + 2.1 - 1.6 = 0, so $\partial L/\partial w = 0$.

The Session 4 normal equation agrees: X^TX rows (4, 10), (10, 30), X^Ty = (23, 67), so b = 1, w = 1.9.

The spreadsheet lab computes the loss for a grid of (w, b) pairs

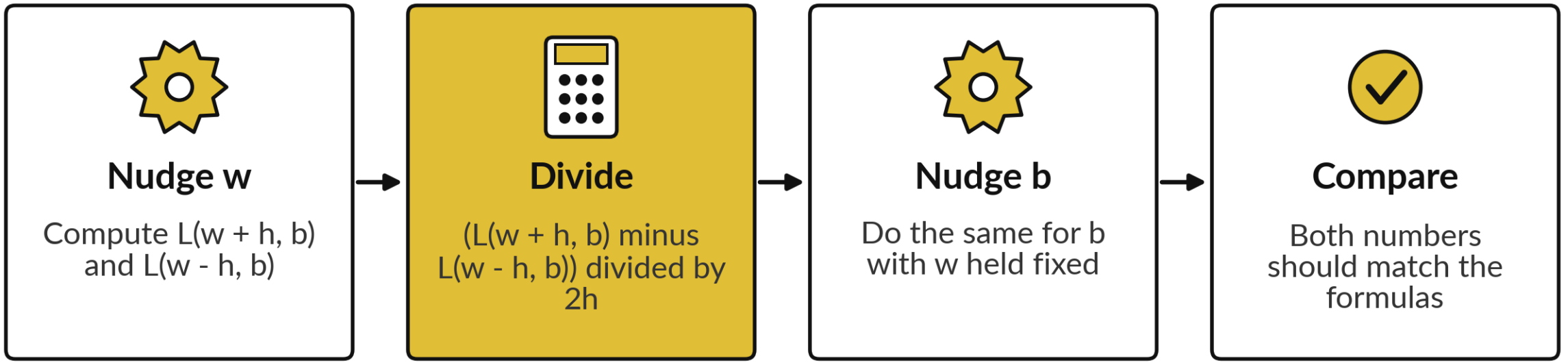
	A	B	C	D	E	F
3	**b \ w**	**0**	**0.5**	**1**	**1.5**	**2**
4	**-1**	50.25	32.875	19.25	9.375	3.25
5	**-0.5**	43.75	27.625	15.25	6.625	1.75
6	**0**	37.75	22.875	11.75	4.375	0.75
7	**0.5**	32.25	18.625	8.75	2.625	0.25
8	**1**	27.25	14.875	6.25	1.375	0.25
9	**1.5**	22.75	11.625	4.25	0.625	0.75

This is the top left of sheet Grid. Each cell holds L(w, b) for the w in row 3 above it and the b in column A beside it.

Start D6 = 11.75 (w = 1, b = 0). The smallest values, 0.25, tie in F7 and F8 (gold).

One formula in B4, with B$3 for w and $A4 for b, copies to every cell.

Check a gradient with numbers: nudge one parameter at a time



With h = 0.0001 at (1, 0): -18.500 for w and -6.500 for b. The formulas gave -18.5 and -6.5.

A gradient has one entry for each parameter, whatever the size of the model



One entry for every parameter of the model

$\theta_1 \ldots \theta_N$: the N parameters

Our line fit: N = 2 (w and b)

Papers write all of a model's numbers together as θ (theta). With N parameters θ_1, θ_2 and so on, the gradient has N entries.

For our line fit, N = 2: θ is (w, b) and the gradient is (∂L/∂w, ∂L/∂b).

Nothing else changes when N grows. Only the length of the list does.

The gradient points uphill, and a partial derivative moves one input only

Walking with the gradient

The gradient points uphill.

To reduce the loss, go the other way.

Use minus the gradient.

Changing both at once

A partial derivative changes one input.

All the others stay fixed.

Nudge w alone, then b alone.

The gradient says where the loss rises. Step the opposite way.

Seven lines of numpy compute both partial derivatives at once

```
import numpy as np
x = np.array([1, 2, 3, 4])
y = np.array([3, 5, 6, 9])
w, b, n = 1, 0, 4
e = b + w * x - y
grad_w = 2 / n * np.sum(x * e)
grad_b = 2 / n * np.sum(e)
print(grad_w, grad_b)
```

```
-18.5 -6.5
```

In plain words

Lines 1 to 4: the data, and the starting values w = 1, b = 0, n = 4.

Line 5: e holds the four errors, prediction minus actual.

Line 6: multiply each error by its x, add them, and multiply by 2 / n. This is ∂L/∂w.

Line 7: add the errors and multiply by 2 / n. This is ∂L/∂b.

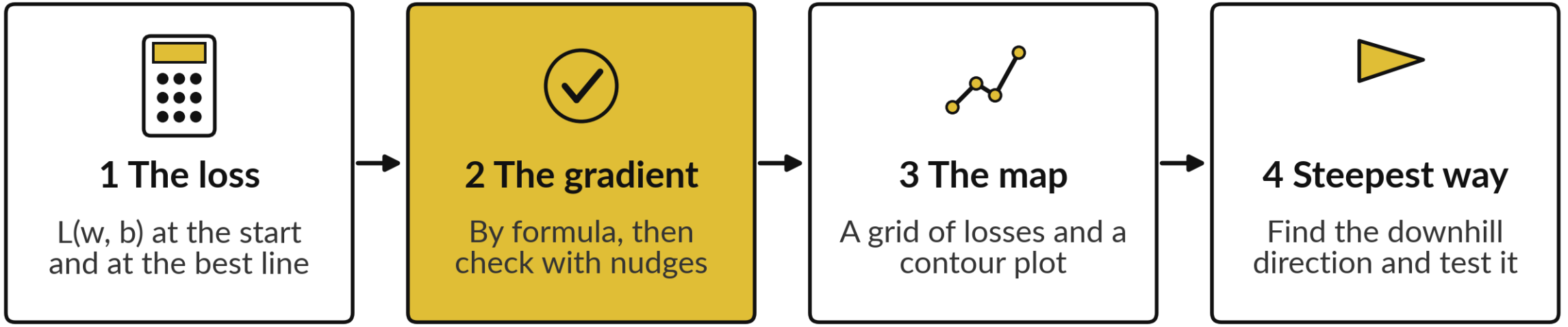
With w = 1 and b = 0 it prints -18.5 and -6.5.



Part 2: the lab

Close the slides. Open the notebook or the workbook.

The lab goes from the loss, to its gradient, to a map, to the steepest way down



Python: Steps 1 to 6 of the notebook. Spreadsheet: sheets Gradient and Grid. In every step predict first, then run, then change one thing.

Your gradient at the start should read (-18.5, -6.5); check the rest here

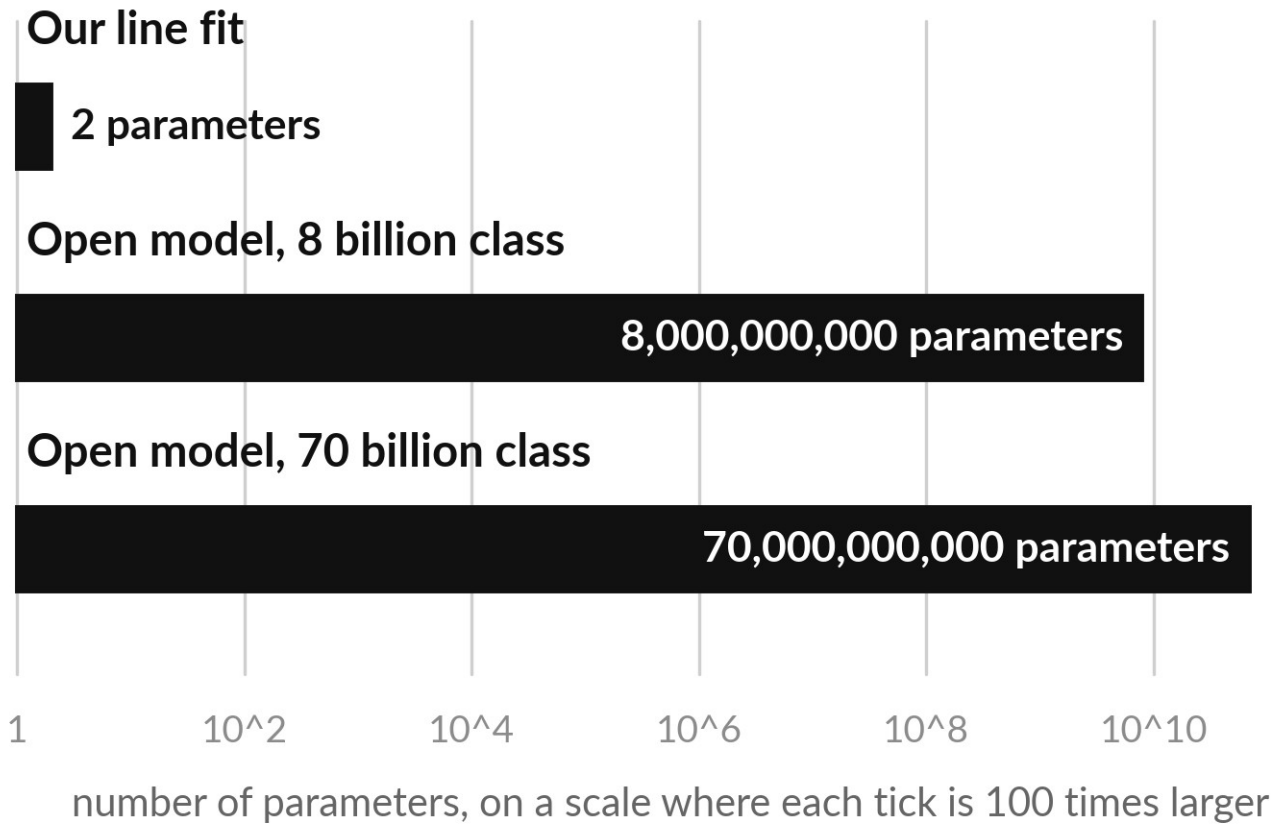
	What you compute	You should see
	Loss L(1, 0) and L(1.9, 1)	11.75 and 0.175
	Gradient at (1, 0): ∂L/∂w and ∂L/∂b	-18.5 and -6.5
	Nudged estimates at (1, 0), h = 0.0001	-18.500 and -6.500
	Gradient at (1.9, 1)	0 and 0 (to 3 decimals)
	Smallest loss on the grid, w = 0 to 3.5, b = -1 to 3, step 0.5	0.25 (two grid points tie)
	Length of the gradient at (1, 0)	19.61
	Steepest downhill direction at (1, 0), as a unit vector	(0.943, 0.331)

If your number differs, stop and ask. Do not change a number to match. Find the step that went wrong.

Four errors that spoil a gradient, and how to fix each one

	What you see	What it means	What to do
	**NameError in Python**	A cell that defines a name was skipped	Run all cells from the top, in order
	**Gradient has the wrong sign**	Errors computed as actual minus prediction	Use prediction minus actual: b + w x - y
	**Both entries equal**	The x multiplier was left out of the w entry	∂L/∂w needs x times e; ∂L/∂b needs e only
	**Grid cells all show the same number**	A cell reference moved when copied	In B4 write w as B$3 and b as $A4, then copy

Where this appears in AI: a gradient with billions of entries

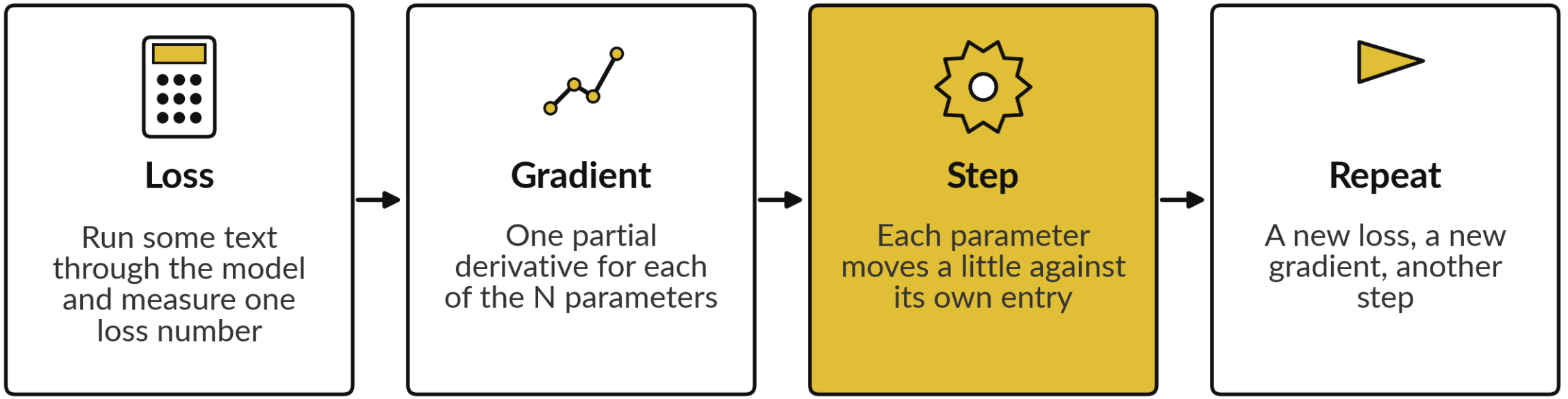


Open models such as Llama 3 list 8 billion and 70 billion parameters. The gradient has one entry for each.

At 2 bytes each, 8 billion entries take 16 GB. Published mixed-precision training also keeps 32-bit copies: several times more memory.

Sizes of ChatGPT, Gemini and Claude are not published.

Where this appears in AI: one training step moves every parameter against its own gradient entry



How the gradient is found through many layers is Sessions 9 and 11 (the backward pass). How big each step is, the learning rate, is Session 10.

Ask an AI assistant for the same two partial derivatives and compare

8 minutes Hands-on, with any assistant

Ask an assistant for the partial derivatives of the mean squared error of y-hat = b + w x at w = 1, b = 0.

Give it x = 1, 2, 3, 4 and y = 3, 5, 6, 9.

Compare with ours: (-18.5, -6.5).

Run your lab gradient code.

If it differs, find the first line that differs.

You keep

One line: its gradient, yours, match or not. Never paste confidential data into an assistant.

Exit check: a partial derivative, a downhill step and the size of a gradient

Question	Short answer
For f(x, y) = x squared times y, what is the partial derivative with respect to y at (3, 2)?	9. Treat x squared as a constant: the slope in y is x squared, which is 9.
The gradient of a loss at some point is (4, -2). In which direction do you step to reduce the loss?	Along minus the gradient: (-4, 2). Decrease the first parameter, increase the second.
How many entries does the gradient of a model with 20 parameters have?	20, one partial derivative for each parameter.

Write your own answer before you read ours. The automatic check counts the lab numbers.

Save the lab and keep the gradient at the start and at the best fit

What you keep from today

Keep your saved lab. Write down the gradient of the loss at the start, (-18.5, -6.5), and the best line, w = 1.9 and b = 1, where the gradient is (0, 0).

Our question today

Many models have billions of adjustable numbers. Which way should each one move to lower the loss?

Next session

Session 9: the chain rule and the Jacobian.

Take away

Saved lab
Notebook or workbook

Gradient at start
(-18.5, -6.5)

Best line
w = 1.9, b = 1

Next session
The chain rule